# SIMATS ENGINEERING
## ASSESSMENT TOOL 2

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA1709**

**CO1-AT2-Constraint-Based Problem Solving**

| | |
|---|---|
| **Register Number** | **192311366** |
| **Name** | **Gondel Prashanth** |

**COURSE OUTCOME COVERED**

***CO1:*** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*( BL3)*

*Assessment Tool Weightage: 20%*

---

**QUESTIONS:3** **Total Marks:30**

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.
   The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

   Constraints:

   - The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

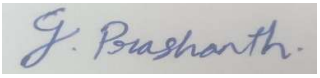
Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

***Code of Conduct:***

*I Gondel Prashnath / 192311366 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***



**RUBRICS FOR CALCULATION:**

| Criterion | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) | Max Marks |
|---|---|---|---|---|---|
| **Criteria** | **Level 4 (Exemplary)** | **Level 3 (Proficient)** | **Level 2 (Developing)** | **Level 1 (Beginning)** | **Max Marks** |
| **Problem Understanding** | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps | Partial understanding; misses key constraints | Fails to identify constraints correctly | 6 |
| **Constraint Analysis** | Analyzes constraints effectively and prioritizes them logically | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization | No meaningful analysis or prioritization | 6 |
| **Solution Development** | Develops optimal and feasible solutions satisfying all constraints | Develops workable solutions with minor limitations | Solutions partially satisfy constraints | Solutions do not meet constraints | 7.5 |

| | | | | | |
|---|---|---|---|---|---|
| **Application of Concepts** | Effectively applies appropriate concepts, methods, or tools | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply relevant concepts | 6 |
| **Justification** | Provides strong justification for decisions with logical reasoning | Provides justification with some clarity | Weak or unclear justification | No justification provided | 4.5 |

1) Problem Setup

- Variables: D1, D2, D3
- Domain (Shifts): Morning (M), Afternoon (A), Night (N)
- Order: M < A < N

Constraints:

1. C1: D1 ≠ N
2. C2: D2 < D3
3. C3: All variables must have different values (Only one doctor per shift)
4. C4: D3 ≠ M

Backtracking Search Process

We will evaluate the variables in the order D1, D2, D3 and attempt to assign shifts in chronological order (Morning, Afternoon, Night).

Step 1: Assigning D1

Try D1 = Morning (M)
Check C1: D1 is not Night. (Valid)
Move to D2.

Step 2: Assigning D2 (Given D1 = M)

Try D2 = Morning (M)
Check C3: D1 is already assigned Morning. (Fails - Backtrack)
Try D2 = Afternoon (A)
Check C3: D1 (M) ≠ D2 (A). (Valid)
Move to D3.

Step 3: Assigning D3 (Given D1 = M, D2 = A)

Try D3 = Morning (M)
Check C3: D1 is already assigned Morning. (Fails - Backtrack)
Try D3 = Afternoon (A)
Check C3: D2 is already assigned Afternoon. (Fails - Backtrack)
Try D3 = Night (N)
Check C3: D1(M) ≠ D2(A) ≠ D3(N). (Valid)
Check C4: D3 is not Morning. (Valid)
Check C2: D2 (Afternoon) < D3 (Night). (Valid)

All doctors are assigned exactly one unique shift, and all constraints are satisfied. The search successfully terminates.

Based on our first successful path through the backtracking search tree:

| Docters | Assigned Shift |
|---|---|
| D1 | Morning |
| D2 | Afternoon |
| D3 | Night |

2) Given that :
Gride Size 5*5
Moves from Start S to Goal G
It move Up, Down, Right, Left
Cost of each move =1
Let's assume the Start is at the top-left (0,0), the Goal is at (2,2), and there is a wall blocking the right side early on.
Step 1: Start Line

- The robot starts at (0,0).
- It guesses it is 4 steps away from the goal.
- To-Do List: [Look at (0,0)]

Step 2: First Move

- The robot can't go Right because of a wall, so it steps Down to (1,0).
- Steps taken: 1. Guessed steps left: 3. Total score: 4.
- To-Do List: [Look at (1,0)]

Step 3: Second Move

- From (1,0), the right side is still blocked. It steps Down again to (2,0).
- Steps taken: 2. Guessed steps left: 2. Total score: 4.
- To-Do List: [Look at (2,0)]

Step 4: Third Move

- Now the robot has cleared the wall. From (2,0), it steps Right to (2,1).
- Steps taken: 3. Guessed steps left: 1. Total score: 4.
- To-Do List: [Look at (2,1)]

Step 5: Goal Reached!

- From (2,1), it steps Right one more time and lands on the Goal (2,2).
- Steps taken: 4. Guessed steps left: 0. Total score: 4.

The Final Answer in Plain English

- Optimal Path: Start → Down → Down → Right → Right → Goal
- Total Cost: 4 steps

3) a) Analysis of Constraints and Decision-Making
Every constraint forces the robot to adapt its decision-making process from a simple puzzle-solver to an adaptable, real-time navigator:

- Movements (Up, Down, Left, Right): This defines the robot's action space. It cannot move diagonally, meaning it must calculate distances using Manhattan distance geometry rather than straight lines.
- Blocked Cells (Obstacles): This restricts the state space. The robot must be capable of recognizing dead ends and rerouting around them.
- Limited Sensing (Partially Observable): This is the most critical constraint. The robot cannot plan a perfect route from the start because it doesn't have a map. It must explore, meaning its decision-making must be iterative—plan, move, observe, and plan again.

b) Solution Approach: Online Agent with Dynamic UCS

1. Initialize the Mental Map: The robot starts with a blank grid in its memory. It assumes all unobserved cells are open and standard cost until proven otherwise.
2. Perceive and Update: The robot uses its sensors to observe the immediate adjacent cells (Up, Down, Left, Right). It updates its internal map, marking any newly discovered obstacles or risky zones.
3. Plan (Using UCS): Based on its current internal map, the robot runs a Uniform Cost Search. UCS expands paths based on lowest cumulative cost, naturally prioritizing safe zones over risky ones. It calculates the cheapest known path to the Goal.
4. Act: The robot executes only the first step of this planned path.
5. Re-evaluate: After taking the step, the robot is in a new cell. It returns to Step 2—sensing its new surroundings, updating its map, and replanning if a new obstacle or risky zone blocks its intended path.
6. Terminate: This cycle continues until the robot reaches the Goal (Survivor).

c) Application of AI Concepts

Intelligent Agents:

An intelligent agent perceives its environment via sensors and acts upon it via actuators to achieve a goal. Here, the robot is a Utility-Based Agent. It doesn't just want to reach the goal (which a simple Goal-Based agent would do); it wants to reach it by maximizing "utility"—specifically, by minimizing the numerical cost associated with risky zones and extra steps.

Rationality:

In AI, rationality does not mean omniscience (knowing everything). A rational agent acts to maximize its expected performance measure given the percept sequence it has seen so far.

- If the robot takes a path that looks safe, but hits a dead end hidden by its limited sensors, the robot is still acting rationally. It made the optimal choice based on the limited information it had at the time, and rationally updated its route once new information was discovered.

Environment Type:

The disaster building can be classified by the following properties:

- Partially Observable: The robot only sees adjacent cells, not the whole grid.
- Deterministic: If the robot chooses to move "Up", it successfully moves "Up" (no random slipping).
- Sequential: The choice to enter a specific room now dictates which rooms are available to enter next.

d) Justification of the Chosen Approach

1. It handles partial observability: Pure UCS or A* expects a full map upfront. By interleaving execution and planning (taking one step, then looking around), the Online Agent safely navigates the unknown without crashing into unmapped obstacles.
2. It guarantees cost-efficiency: Unlike Depth-First Search (which just finds any path) or Breadth-First Search (which finds the path with the fewest steps), UCS actively evaluates the cumulative cost. It will correctly route the robot around a risky zone if a slightly longer, safer detour exists.
3. It prevents infinite loops: By constantly updating its internal map, the robot "remembers" where it has been and where the dead ends are, ensuring it eventually finds the survivor rather than pacing back and forth in a collapsed hallway.